

Aula 4 — Servidor, Docker, Kamal e deploy

Duração: ~3h · **Você sai daqui com:** a sua API no ar, em `https://seunome.test`, servida por um servidor Linux de verdade que você mesmo subiu, com HTTPS e deploy por um comando. **Gabarito:**

```
cd ~/capacitacao-gabarito && git checkout aula-04
```

Este roteiro é a versão para a turma do guia de infraestrutura do `seem-backend`. As decisões são as mesmas; a máquina é menor, e hoje ela mora no seu notebook.

O último dia

Você tem uma API com cinco fluxos de autenticação, testada, rodando na sua máquina. **Hoje ela sai da sua máquina** e passa a rodar num servidor Linux que você mesmo vai provisionar: firewall, Docker, usuário, chave SSH, HTTPS.

Esta é a aula mais diferente das quatro. Nas outras você escreveu Ruby; hoje você opera uma máquina. São ferramentas novas, comandos novos e um vocabulário inteiro novo em três horas — **é normal se sentir mais perdido hoje do que nos outros encontros**. Não é você indo mal; é assunto novo mesmo, e a maior parte dele se aprende repetindo.

Duas coisas que ajudam:

Preste atenção em qual máquina você está. Metade da confusão do dia é rodar na máquina errada. O prompt te diz: `você@seu-notebook` ou `ubuntu@servidor`. Olhe antes de cada comando.

Um passo mal feito só aparece três passos depois. Por isso toda prática tem uma linha **Confere** — não siga com ela vermelha, mesmo que pareça que dá.

E o fecho, para você já saber onde vai chegar: no fim do dia, três comandos `curl` vão te mostrar, na tela, a diferença entre criptografia e confiança. É a Prática 7, e vale a aula.

1. Onde o seu código vai rodar

O servidor de hoje é uma VM no seu notebook

Você vai criar uma **máquina virtual**: um computador inteiro — kernel, disco, rede, usuários — simulado por software dentro do seu. Ela roda Ubuntu Server, tem IP próprio, e você entra nela por SSH exatamente como entraria numa máquina alugada do outro lado do mundo.

Por que numa VM e não direto no seu sistema? Porque um servidor tem que ser descartável. Você vai instalar coisa, errar configuração de SSH, encher o disco. Numa VM, o conserto é `multipass delete servidor && multipass launch`. No seu notebook, o conserto é o seu sábado.

E por que não só Docker, sem VM? Porque containers compartilham o kernel do host: você não tem usuário, firewall, `systemd`, `sshd`. Metade desta aula é exatamente o que uma VM tem e um container não.

O que muda quando o servidor é alugado

	A VM de hoje	Uma VPS alugada
Onde roda	seu notebook	datacenter de um provedor
IP	privado, só a sua máquina alcança	público, o mundo inteiro alcança
Firewall	<code>ufw</code> , dentro da VM	<code>ufw</code> e o firewall do provedor
Certificado	autoassinado, você confia nele na mão	emitido por uma CA pública
Custo	zero	por hora ligada
Quem invade	ninguém, não está exposta	bots, o dia todo, desde o primeiro minuto

Todo o resto é idêntico: o Ubuntu, o Docker, o Kamal, o `deploy.yml`, os segredos, o zero-downtime. É por isso que aprender aqui vale — quando você trocar o IP privado por um público, o que muda é uma variável.

Na seção **11** você vê o percurso na nuvem, com Azure e Cloudflare, e o apêndice `apendice-azure.md` tem o passo a passo completo para você fazer em casa.

VPS, PaaS e Kubernetes

VPS = *Virtual Private Server*. Um computador virtual, dentro do servidor físico de um provedor, que é seu: você tem acesso root, escolhe o sistema, instala o que quiser. É a mesma coisa que a sua VM de hoje — alugada.

Opção	O que você controla	O que você opera
Hospedagem compartilhada	quase nada	nada
PaaS (Heroku, Render, Fly)	o app	nada — mas paga mais e obedece as regras deles
VPS	tudo	tudo: SO, atualizações, firewall, banco
Kubernetes	tudo, em escala	muito mais

Por que VM com containers, e não Kubernetes

O `seem-backend` atende ~500 usuários num evento de uma semana. Kubernetes adicionaria custo e operação sem resolver um problema que existe. A escolha foi:

- menos peças para monitorar;
- deploy reproduzível com Docker e Kamal;
- banco e aplicação perto um do outro;
- recuperação simples numa VM substituta;
- **evoluir só quando uma métrica real pedir.**

Escolher a ferramenta grande antes do problema grande é a forma mais cara de errar.

O que isso custa, e é honesto admitir: uma VM é ponto único de falha. Volume Docker não é backup. O Postgres não é gerenciado — quem cuida dele é você.

2. Criar a VM

Vamos usar o **Multipass**, da Canonical: ele baixa uma imagem oficial do Ubuntu Server, cria a VM e te devolve um IP. Um comando.

Instalar

```
# Linux
sudo snap install multipass

# macOS
brew install --cask multipass

# Windows (PowerShell como administrador)
winget install Canonical.Multipass
```

Confira:

```
multipass version
```

Windows + WSL2: o Multipass roda no Windows, e o seu Rails roda dentro do WSL2 — são duas máquinas virtuais diferentes, e por padrão uma não enxerga a outra. Há duas correções, uma delas funcionando em qualquer Windows: estão em `00-preparacao.md` e em `troubleshooting.md`. **Faça antes da aula**, e confirme com o `scripts/checar-servidor.sh`.

Antes: duas chaves, um par

Criptografia assimétrica usa duas chaves que se completam. A **pública** você espalha; a **privada** nunca sai da sua máquina. O que uma fecha, só a outra abre.

Daí saem duas coisas diferentes:

- **sigilo** — eu fecho com a *sua* pública, e só você abre;
- **assinatura** — eu fecho com a *minha* privada, e todo mundo confere que fui eu.

É assim que o SSH funciona. Você põe a sua chave pública no servidor (`~/.ssh/authorized_keys`). Ao conectar, o servidor manda um desafio; você responde assinando com a privada; o servidor confere com a pública que já tinha. **A senha nunca trafega — nem existe.**

E é por isso que perder a chave privada é perder o acesso à máquina.

Compare com o JWT da Aula 3: lá a assinatura usa uma chave **simétrica** (HS256) — a mesma chave assina e confere, porque quem assina e quem confere são o mesmo servidor.

O par de chaves da capacitação

Não reaproveite a chave do GitHub. Uma chave por finalidade: dá para revogar uma sem derrubar a outra.

```
ssh-keygen -t ed25519 -f ~/.ssh/capacita -C "capacita-servidor" -N ""
chmod 600 ~/.ssh/capacita
```

`chmod 600` = só você lê. O SSH **recusa** usar uma chave com permissão frouxa.

Chave privada não vai por e-mail, não vai por WhatsApp, não vai para o Git. Nunca.

`cloud-init`: a máquina já nasce configurada

`cloud-init` é o padrão que praticamente todo provedor de nuvem usa para configurar uma VM no primeiro boot: usuários, chaves, pacotes. Aqui ele serve para plantar a sua chave pública antes mesmo de a máquina existir.

```
cd ~/automic_auth_api

cat > cloud-init.yaml <<EOF
#cloud-config
ssh_authorized_keys:
  - $(cat ~/.ssh/capacita.pub)
EOF
```

Repare no `$(cat ...)`: quem entra no arquivo é a chave **pública**. A privada continua onde sempre esteve.

Subir

```
multipass launch 24.04 --name servidor \  
  --cpus 2 --memory 2G --disk 10G \  
  --cloud-init cloud-init.yaml
```

Descubra o IP:

```
multipass info servidor
```

```
Name:      servidor  
State:     Running  
IPv4:      10.161.42.87  
Release:   Ubuntu 24.04.3 LTS
```

Guarde esse IP: é o seu `SERVER_IP` daqui para frente. Ele pode mudar se você desligar e ligar a VM — confira com `multipass info` sempre que algo parar de conectar.

Primeiro acesso

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP
```

O prompt vira `ubuntu@servidor`. **Você está dentro de outro computador.**

Não conectou? Antes de investigar, rode o diagnóstico — ele testa a porta, testa a chave, e imprime a saída do seu caso:

```
cd ~/capacitacao-gabarito && ./scripts/checar-servidor.sh SEU_IP
```

Se você resolveu o caso do Windows com encaminhamento de porta, o seu `SEU_IP` é o do **Windows** e o SSH está na 2222: `ssh -p 2222 -i ~/.ssh/capacita ubuntu@SEU_IP`. Vale para todos os comandos `ssh` desta apostila, e para o `SSH_PORT` do seu `.env`.

O `multipass shell servidor` também entra, e é o atalho de emergência quando você quebra o SSH. Mas use o `ssh` no dia a dia: é ele que você teria numa VPS, e é ele que o Kamal usa.

3. Preparar o Ubuntu

Atualizar

```
sudo apt update && sudo apt upgrade -y  
test -f /var/run/reboot-required && sudo reboot
```

Firewall

Uma máquina só deve aceitar conexão no que ela realmente serve. O Ubuntu traz o **ufw** (*Uncomplicated Firewall*), que é uma casca amigável sobre as regras do kernel.

```
sudo ufw default deny incoming      # nada entra...
sudo ufw default allow outgoing     # ...mas a máquina pode sair
sudo ufw allow 22/tcp               # SSH
sudo ufw allow 80/tcp               # HTTP
sudo ufw allow 443/tcp              # HTTPS
sudo ufw enable
sudo ufw status verbose
```

Seja honesto sobre o que isso faz hoje: a sua VM não está na internet, então esse firewall não está te protegendo de bot nenhum. Você está aprendendo o gesto — e numa VPS ele é literalmente o que separa a sua máquina de um scanner que bate na porta 22 a cada poucos segundos, desde o primeiro minuto em que ela existe.

Numa VPS há **dois** firewalls: o `ufw` dentro da máquina e o do provedor, fora dela (na Azure chama-se NSG). Você configura os dois, e o de fora é o que importa mais — porque o pacote nem chega à sua máquina.

Docker, do repositório oficial

O `apt install docker.io` do Ubuntu instala uma versão velha. Use o repositório da Docker:

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Libere o Docker para o `ubuntu`:

```
sudo usermod -aG docker ubuntu
exit
```

Saia e entre de novo — grupo só vale em sessão nova.

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP
docker run --rm hello-world
```

Root e permissões

`root` é o usuário que pode tudo — sem "tem certeza?". Você trabalha como `ubuntu` e chama `sudo` quando precisa.

Todo arquivo tem dono, grupo e três permissões: ler, escrever, executar.

```
chmod 600 arquivo    # dono lê e escreve; mais ninguém vê nada
chmod 700 pasta      # dono entra; mais ninguém
```

O SSH **exige** `600` numa chave privada — com permissão mais frouxa ele recusa usar o arquivo.

E nunca rode a aplicação como root: o Dockerfile já cria um usuário `rails` justamente para isso.

Swap

Do seu notebook, **fora** da sessão SSH:

```
cd ~/capacitacao-gabarito
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'sudo bash -s' < scripts/server-swap.sh
```

Rails + Postgres juntos numa máquina pequena estouram a RAM, e o kernel mata o processo que estiver na frente (*OOM killer*), com uma mensagem que não explica nada. 2 GiB de swap resolvem.

A sua VM tem 2 GiB e provavelmente não vai precisar. A VPS de US\$5/mês que você vai alugar depois tem 1 GiB e vai. É um comando, e é melhor já saber qual é.

Endurecer o SSH

Mantenha a sessão atual aberta enquanto testa em outro terminal.

```
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
EOF

sudo sshd -t          # valida a sintaxe ANTES de recarregar
sudo systemctl reload ssh
```

Em outro terminal:

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'echo OK'
```

Só feche a primeira sessão depois que isso responder. Errar a config do SSH com uma sessão só aberta é o jeito clássico de perder acesso à máquina.

Aqui você tem uma rede de segurança que uma VPS não te dá: `multipass shell servidor` entra sem passar pelo `ssh`. Use o hábito certo mesmo assim — o dia em que a máquina for alugada, essa porta não existe.

4. Docker

Imagem × container

Imagem é a receita: o sistema, o Ruby, as gems, o seu código, congelados. **Container** é o bolo: uma instância rodando. Uma imagem, muitos containers.

O Dockerfile do Rails

O Rails 8 gera um Dockerfile de produção pronto. Vale ler:

```
FROM ruby:3.4.9-slim AS base
# ...

FROM base AS build
RUN apt-get install -y build-essential libpq-dev ...
COPY Gemfile Gemfile.lock ./
RUN bundle install
COPY . .

FROM base
RUN groupadd --system --gid 1000 rails && useradd rails --uid 1000 --gid 1000
USER 1000:1000
COPY --from=build "${BUNDLE_PATH}" "${BUNDLE_PATH}"
COPY --from=build /rails /rails
```

Duas decisões importantes:

Multi-stage. O estágio `build` instala compilador e headers para compilar as gems nativas. A imagem final copia só o resultado. Compilador não vai para produção: menos peso e menos ferramenta à mão de quem invadir.

Usuário não-root. Se alguém escapar da aplicação, cai num usuário sem privilégio. É uma linha que muda o tamanho do estrago.

`.dockerignore`

Diz o que **não** entra na imagem: `.git`, `log/`, `tmp/`, `node_modules`. Sem ele a imagem fica gorda e — pior — o histórico do Git vai junto.

Registry

O registry é o "GitHub das imagens". Usamos o **ghcr.io** (GitHub Container Registry): já vem com a sua conta do GitHub.

O fluxo desta aula:

```
seu notebook builda → empurra a imagem para o ghcr.io → a VM puxa e roda
```

A imagem não viaja pela sua rede local: ela sobe para o GitHub e desce de lá. É exatamente o que aconteceria com uma VPS do outro lado do mundo, e é o que faz o mesmo `deploy.yml` funcionar nos dois casos.

O token do registry

Fora do GitHub Actions não existe `GITHUB_TOKEN`. Crie um **Personal Access Token (classic)**:

GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**, com os escopos `write:packages` e `read:packages`.

Guarde: é o seu `KAMAL_REGISTRY_PASSWORD`. Ele é mostrado **uma vez**.

Um token com escopo de pacote e nada mais. É o mesmo princípio da chave SSH separada: quando vazar — e um dia vaza — o estrago tem tamanho.

Arquitetura

O Kamal builda a imagem no seu notebook e ela roda na VM. As duas precisam da **mesma arquitetura de processador**:

Seu notebook	SERVER_ARCH
Intel ou AMD	amd64
Mac com chip M1/M2/M3/M4	arm64

Como a VM roda no seu próprio hardware, a arquitetura dela é a sua. Descubra com:

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'dpkg --print-architecture'
```

5. Kamal

O Kamal (feito pela mesma turma do Rails) faz *zero-downtime deploy* com Docker, via SSH. Sem agente, sem painel: ele conecta na sua máquina e roda `docker`.

O que ele faz num `kamal deploy`:

1. builda a imagem e empurra para o registry
2. conecta na VM por SSH
3. puxa a imagem
4. sobe o container novo **ao lado** do antigo
5. espera o health check do novo passar
6. manda o tráfego para o novo e derruba o antigo

Falhou o health check? Ele não troca. O antigo continua servindo.

`config/deploy.yml`, por partes

```
service: automic-auth-api
image: <%= ENV.fetch("GHC_USER") %>/automic-auth-api
```

O arquivo é ERB: dá para usar `ENV`. É por isso que este `deploy.yml` é **igual para todo mundo da turma** — e é o mesmo que serviria numa VPS. O que muda são as variáveis.

```
ssh:
  user: ubuntu
  run_directory: /home/ubuntu/.kamal
```

Sem login root, então o Kamal precisa de um lugar para gravar lock e auditoria.

```
servers:
  web:
    hosts:
      - <%= ENV.fetch("SERVER_IP") %>
```

`SERVER_IP` é o IP da sua VM hoje. Numa VPS, é o IP público. Uma variável.

O bloco `ssh` também lê `SSH_PORT`, com padrão 22 — é o que faz o Kamal funcionar sem mudança nenhuma para quem alcança a VM por encaminhamento de porta.

```
env:
  clear:
    SOLID_QUEUE_IN_PUMA: "true"
    WEB_CONCURRENCY: "1"
    DB_HOST: automic-auth-api-db
  secret:
    - RAILS_MASTER_KEY
    - JWT_SECRET
```

- `clear` vai como texto no comando `docker run`. Configuração, não segredo.
- `secret` vai por um arquivo de ambiente, com permissão restrita no servidor.

Colocar `JWT_SECRET` no `clear` seria o mesmo que publicá-lo: ele apareceria em `docker inspect` e no histórico do shell.

`DB_HOST: automic-auth-api-db` é o **nome do container** do banco. Containers na mesma rede Docker se enxergam por nome — não precisa de IP.

```
accessories:
  db:
    image: postgres:16
    directories:
      - data:/var/lib/postgresql/data
```

Accessory é um container que o Kamal gerencia mas não faz deploy junto com o app. O `directories` cria o volume: é ele que faz os dados sobreviverem a deploy e a rebuild.

`kamal deploy` **não** sobe accessories. Depois de mudar env de accessory: `kamal accessory reboot db`.

As três camadas de segredo

```
o seu shell (source .env)
  ↓ variáveis de ambiente na sua sessão
.kamal/secrets
  ↓ referencia as variáveis – nenhum valor aqui, por isso vai para o Git
config/deploy.yml (env.secret)
  ↓ o Kamal injeta no container
ENV["JWT_SECRET"] no Rails
```

Nenhum valor real toca o repositório em nenhum ponto. Num pipeline de CI a primeira camada vira os Secrets do GitHub Actions; **as outras três não mudam nada**.

6. Segredos do Rails

`credentials` e `master.key`

```
bin/rails credentials:edit
```

O Rails abre um YAML no editor, e ao salvar grava `config/credentials.yml.enc` — criptografado, seguro no Git. A chave que decripta é `config/master.key`, que **nunca** vai para o Git (já está no `.gitignore`).

Em produção, `master.key` vira a variável `RAILS_MASTER_KEY`.

Antes: o que é uma variável de ambiente

Um par `nome=valor` que o sistema operacional entrega ao processo quando ele sobe.

```
export JWT_SECRET=abc123
```

E o programa lê com `ENV["JWT_SECRET"]`.

Por que não deixar o valor no código? Porque o código vai para o Git. A ideia é: mesmo código, ambientes diferentes, valores diferentes. É um dos [12 fatores](#), e é como o Kamal injeta segredo no container.

Credentials × ENV

	Quando
Credentials	segredo estável, que é do app: chave de API de terceiro
ENV	o que muda por ambiente ou por máquina: senha do banco, host de SMTP

Este projeto usa ENV para quase tudo, porque quem provisiona (Kamal) já sabe injetar.

Gere o `JWT_SECRET`:

```
bin/rails secret
```

O seu `.env`

O Kamal lê o `.kamal/secrets`, que só **referencia** variáveis do seu shell. Quem põe valor nelas é você. Copie o exemplo e preencha:

```
cd ~/automic_auth_api
cp .env.example .env
$EDITOR .env
```

E carregue antes de qualquer comando do Kamal:

```
source .env
```

O `.env` está no `.gitignore` — confira antes de commitar. **Este é o arquivo que não pode vazar.**

7. HTTPS na sua máquina

HTTPS é HTTP dentro de um túnel

O HTTP da Aula 1 é **texto puro**: quem estiver no caminho — o roteador do café, o provedor — lê tudo, inclusive a senha. O **TLS** embrulha esse texto num túnel criptografado.

Mesmo protocolo, mesmos verbos, mesmos cabeçalhos, só que fechado. O "S" de HTTPS é isso, e nada além disso.

O handshake, em três passos

1. O servidor apresenta o **certificado** dele.
2. O cliente confere se confia em **quem assinou** aquele certificado.
3. Os dois combinam uma chave temporária e conversam com ela.

O par de chaves assimétricas só serve para o passo 3. Depois disso a conversa usa criptografia **simétrica**, que é muito mais rápida.

Certificado e autoridade

Um certificado diz: *"esta chave pública pertence a este domínio"* — e vem **assinado por uma Autoridade Certificadora (CA)**.

O seu navegador já nasce com uma lista de CAs em que confia. Confia na CA → confia em quem ela assinou. É uma cadeia.

- **Autoassinado** é você jurando que é você.
- **Let's Encrypt** é uma CA pública e gratuita, em que os navegadores confiam.

Para conseguir um certificado de CA pública é preciso **provar que o domínio é seu** — e você não tem domínio nenhum apontando para a sua VM. Então hoje o certificado é autoassinado, e você vai ver, com os próprios olhos, o que isso significa.

Proxy reverso

Um proxy comum fica na frente do **cliente**. Um proxy **reverso** fica na frente do **servidor**: recebe tudo na 443, termina o TLS, e repassa para a aplicação em HTTP interno.

Assim o Rails não precisa saber nada de certificado:

```
seu navegador → kamal-proxy (na VM) → Rails
```

É por isso que `config.assume_ssl = true`: ele avisa o Rails de que o "http" que chegou já veio de um "https" lá fora, e o Rails para de montar URLs erradas.

Gerar o certificado

Do seu notebook, na raiz do projeto:

```
openssl req -x509 -newkey rsa:2048 -sha256 -days 365 -nodes \
  -keyout tls/capacita-key.pem -out tls/capacita-cert.pem \
  -subj "/CN=seunome.test" \
  -addext "subjectAltName=DNS:seunome.test"
```

O `subjectAltName` não é opcional. Cliente moderno nenhum olha só o `CN` — sem SAN, o certificado é inválido mesmo estando certo.

E ponha o conteúdo nas variáveis, no seu `.env`:

```
KAMAL_PROXY_SSL_CERTIFICATE="$(cat tls/capacita-cert.pem)"
KAMAL_PROXY_SSL_PRIVATE_KEY="$(cat tls/capacita-key.pem)"
export KAMAL_PROXY_SSL_CERTIFICATE KAMAL_PROXY_SSL_PRIVATE_KEY
```

`.test` e o `/etc/hosts`

`seunome.test` não existe em DNS nenhum, e nunca vai existir: `.test` é um domínio **reservado para testes** justamente para isso ([RFC 6761](#)). Ninguém consegue registrar, então ele nunca vai colidir com um site de verdade.

Quem resolve esse nome é o seu próprio `/etc/hosts`, que o sistema consulta **antes** de perguntar ao DNS:

```
echo "SEU_IP seunome.test" | sudo tee -a /etc/hosts
```

```
getent hosts seunome.test # tem que devolver o IP da VM
```

Windows: repita no arquivo `C:\Windows\System32\drivers\etc\hosts`, com o Bloco de Notas aberto como administrador. É esse que o navegador do Windows consulta — o `/etc/hosts` do WSL2 vale só dentro do WSL2.

Autoassinado, na prática

Depois do deploy (seção 9), três comandos que valem a aula inteira.

Antes de rodar o primeiro, decida: o seu servidor está servindo HTTPS de verdade, com um certificado que você mesmo gerou. O `curl` vai funcionar ou vai reclamar? E se reclamar, vai ser por causa da criptografia ou por outra coisa?

```
curl https://seunome.test/api/v1/status
```

```
curl: (60) SSL certificate problem: self signed certificate
```

Isso é o TLS funcionando, não falhando. O túnel subiu; o que falhou foi a *confiança*. O cliente não conhece quem assinou.

```
curl -k https://seunome.test/api/v1/status
```

O `-k` é "criptografa, mas não confere quem é". Funciona, e é exatamente o que você **não** faz em produção: sem verificar o certificado, qualquer um no meio do caminho pode se passar pelo servidor.

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

Aqui você não desligou nada: você **disse ao curl em quem confiar**. E é isso que uma CA é — alguém em quem o seu sistema já decidiu confiar, de fábrica. Let's Encrypt não tem mágica nenhuma que o seu `openssl` não tenha; tem o navegador do mundo inteiro com a chave dela na lista.

Abra `https://seunome.test/api/v1/status` no navegador também. O aviso vermelho é o mesmo fato, dito para um humano.

8. GitHub Actions: o portão

`.github/workflows/ci.yml` roda em todo push e todo pull request:

- `scan_ruby` — Brakeman (segurança estática) e bundler-audit (CVE em gems)
- `lint` — RuboCop
- `test` — `bin/rails test` contra um Postgres descartável

Se qualquer um falhar, o código não entra na `main`.

Por que isso importa aqui, se o deploy é manual? Porque o CI não depende de onde o servidor mora. Ele é o portão: garante que o que você está prestes a colocar no ar compila, passa nos testes e não tem CVE conhecida. Um servidor sem CI publica bug automaticamente; um CI sem servidor ainda te avisa antes.

O deploy automático a cada push é o passo seguinte, e só faz sentido quando o servidor tem IP público — o runner do GitHub não alcança um IP dentro do seu notebook. Ele está pronto em `.github/workflows/deploy.yml`, e a seção **11** explica o que ele faz.

9. O primeiro deploy

Primeiro, valide sem tocar em nada:

```
cd ~/automic_auth_api
source .env
bundle exec kamal config
```

Sai um YAML grande, sem erro. Se reclamar de variável faltando, a mensagem diz exatamente qual.

Agora:

```
bundle exec kamal setup
```

`setup` instala o Docker se faltar, sobe os accessories e faz o primeiro deploy. **Só na primeira vez** — depois é `kamal deploy`.

Deu certo:

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

```
{"status":"ok","service":"automic-auth-api","environment":"production"}
```

O seu código está rodando dentro de um container, num Ubuntu que você provisionou, atrás de um proxy TLS, com um Postgres com volume. Em produção, é isto.

Daqui para frente, todo deploy é:

```
source .env && bundle exec kamal deploy
```

10. Operar

```
source .env

kamal app logs -f           # logs ao vivo
kamal app exec --interactive --reuse "bin/rails console"
kamal app exec "bin/rails db:migrate"
kamal app details           # o que está rodando
kamal accessory reboot db   # depois de mudar env do banco
kamal rollback              # volta para a versão anterior
```

Na VM:

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP
docker ps
free -h
df -h
```

Backup

Volume Docker não é backup. Se a VM sumir, o volume some junto — e aqui a VM some com um `multipass delete`, o que é uma demonstração barata de um susto caro.

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP \
  'docker exec automic-auth-api-db pg_dump -U automic_auth_api automic_auth_api_production' \
  | gzip > backup-$(date +%F).sql.gz
```


E, o que quase ninguém faz: **restaure num banco de teste**. Backup que nunca foi restaurado não é backup, é esperança.

Desligar e voltar

```
multipass stop servidor      # libera RAM e CPU do notebook
multipass start servidor     # volta; confira o IP com multipass info
multipass delete servidor && multipass purge  # apaga de vez
```

O IP pode mudar entre um `stop` e um `start`. Quando mudar: atualize o `SERVER_IP` no `.env` e a linha no `/etc/hosts`.

11. E na nuvem de verdade

Tudo o que você fez hoje vale numa VPS sem mudar de forma. O que **entra** são quatro coisas, e todas existem porque agora a máquina está exposta ao mundo:

	Hoje	Numa VPS
A máquina	<code>multipass launch</code>	portal do provedor: região, tamanho, imagem, cota
Firewall	<code>ufw</code>	<code>ufw</code> mais o firewall do provedor (na Azure, o NSG), com a 22 aberta só para o seu IP
Nome	<code>/etc/hosts</code>	DNS de verdade, num domínio seu (usamos Cloudflare)
Certificado	autoassinado	emitido por uma CA — Let's Encrypt, ou Origin CA se houver Cloudflare na frente
Deploy	<code>kamal deploy</code> no seu terminal	GitHub Actions, autenticando por OIDC e abrindo a porta 22 por um minuto

O `.github/workflows/deploy.yml` deste repositório é esse pipeline, pronto. A sequência dele:

1. autentica na nuvem (OIDC)
2. descobre o próprio IP público
3. cria uma regra no firewall liberando a 22 só para esse IP/32
4. faz o deploy
5. APAGA a regra — mesmo se o deploy falhar

O passo 5 tem `if: always()`. **Deixar a 22 aberta é o erro que transforma um deploy ruim num incidente de segurança.**

E o **OIDC** é o motivo de não haver senha nenhuma nisso: em vez de guardar um segredo de longa duração no GitHub, o GitHub emite a cada execução um token curto que diz *"sou o workflow do repositório X, na branch `main`"*, e a nuvem devolve um acesso temporário.

Isto é a demonstração de hoje, não o exercício. O passo a passo completo — criar a VM na Azure, configurar o NSG, o DNS no Cloudflare, o certificado Origin CA e a credencial federada do OIDC — está em `apendice-azure.md`, para você fazer em casa com a [Azure for Students](#) (US\$100 de crédito, sem cartão).

As práticas da aula

Oito práticas, cada uma logo depois do bloco que a explica. Nesta aula um passo mal feito só aparece três passos depois — então **não siga com uma conferência vermelha**.

#	O que	Depois de
1	Subir a VM e entrar nela	seção 2
2	O Ubuntu: firewall, Docker, swap, SSH	seção 3
3	Os arquivos de deploy e o token do registry	seção 4
4	O <code>.env</code>	seção 6
5	O certificado e o <code>/etc/hosts</code>	seção 7
6	O primeiro deploy	seção 9
7	O certificado, com os olhos	seção 9
8	O sistema inteiro, e o backup	seção 10

Prática 1 — Subir a VM e entrar nela

~20 minutos, sendo boa parte espera de download. **Comece o `launch` antes da teoria de chaves** e deixe baixando.

```
ssh-keygen -t ed25519 -f ~/.ssh/capacita -C "capacita-servidor" -N ""
chmod 600 ~/.ssh/capacita

cd ~/automic_auth_api
cat > cloud-init.yaml <<EOF
#cloud-config
ssh_authorized_keys:
  - $(cat ~/.ssh/capacita.pub)
EOF

multipass launch 24.04 --name servidor --cpus 2 --memory 2G --disk 10G \
  --cloud-init cloud-init.yaml
multipass info servidor          # anote o IPv4
ssh -i ~/.ssh/capacita ubuntu@SEU_IP
```

Confere: o prompt mudou para `ubuntu@servidor`. Se não conectou, o diagnóstico responde antes de você investigar:

```
cd ~/capacitacao-gabarito && ./scripts/checar-servidor.sh SEU_IP
```

Se der errado

Hoje você vai encontrar mais erros do que nos outros três encontros somados, e a maioria não tem nada a ver com programar: é rede, permissão e ferramenta nova. **É assim para todo mundo, sempre.** Quem trabalha com infraestrutura passa boa parte do tempo exatamente aqui — a diferença é que já reconhece os erros de vista. Você está começando esse repertório hoje.

Erro	Causa	Saída
<code>multipass: command not found</code>	não instalado	volte a <code>00-preparacao.md</code>
<code>launch failed: ... virtualization</code>	virtualização desligada na BIOS/UEFI	procure <code>VT-x</code> , <code>AMD-V</code> ou <code>SVM</code> e ligue
parado em "Retrieving image"	baixando ~500 MB	espere; se travar, <code>multipass delete servidor && multipass purge</code> refaça
<code>ssh</code> dá timeout, e você usa Windows	WSL2 não enxerga a VM	as duas correções estão em <code>00-preparacao.md</code> ; o script diz qual é a
<code>Permission denied (publickey)</code>	permissão frouxa na chave	<code>chmod 600 ~/.ssh/capacita</code>
<code>Permission denied</code> mesmo com <code>chmod</code>	a chave pública não entrou na VM	<code>multipass exec servidor -- cat /home/ubuntu/.ssh/authorized_k</code> — se vazio, o <code>cloud-init.yaml</code> tem o caminho em vez do conteúdo
<code>Too many authentication failures</code>	o SSH tentou todas as chaves do agente	acrescente <code>-o IdentitiesOnly=yes</code>
o IP mudou do nada	houve <code>stop / start</code>	<code>multipass info servidor</code> e atualize

Prática 2 — O Ubuntu: firewall, Docker, swap, SSH

~30 minutos. Tudo dentro da VM, exceto o swap.

```
sudo apt update && sudo apt upgrade -y

sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp && sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
sudo ufw enable
```

Instale o Docker pelo repositório oficial (seção **3** desta apostila), depois:

```
sudo usermod -aG docker ubuntu
exit
```

Saia e entre de novo — grupo só vale em sessão nova. Do seu notebook:

```
cd ~/capacitacao-gabarito
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'sudo bash -s' < scripts/server-swap.sh
```

E, de volta dentro da VM, endureça o SSH (seção 3) — **com uma segunda sessão aberta para testar**.

Confere:

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'docker run --rm hello-world && free -h && sudo ufw
status'
```

"Hello from Docker!", uma linha `Swap:` com 2,0Gi, e o `ufw` como `active`.

Se der errado

Erro	Causa	Saída
<code>permission denied</code> no <code>docker</code> dentro da VM	não saiu e entrou depois do <code>usermod</code>	<code>exit</code> e <code>ssh</code> de novo — reabrir aba não basta
o <code>ufw enable</code> derrubou o seu SSH	você liberou a 22 depois de habilitar	<code>multipass shell servidor</code> entra sem SSH; libere a 22 e saia
<code>Unable to locate package docker-ce</code>	o repositório da Docker não entrou	refaça o bloco do <code>tee /etc/apt/sources.list.d/docker.sources</code> inteiro
<code>NO_PUBKEY</code> no <code>apt update</code>	a chave GPG não foi baixada	refaça o <code>curl ... docker.asc</code> e o <code>chmod a+r</code>
o swap não aparece no <code>free -h</code>	o script rodou sem <code>sudo</code>	o comando tem <code>'sudo bash -s'</code> ; confira
perdi o SSH depois do hardening	erro na config	<code>multipass shell servidor</code> , conserte <code>/etc/ssh/sshd_config.d/99-hardening.conf</code> , <code>sudo sshd -t</code> , <code>sudo systemctl reload ssh</code>
<code>sudo sshd -t</code> reclama	erro de sintaxe	ele diz a linha; conserte antes do reload

Prática 3 — Os arquivos de deploy e o token do registry

~15 minutos.

```
cd ~/capacitacao-gabarito && git checkout aula-04
rsync -aR config/deploy.yml config/environments/production.rb \
    .kamal scripts .github .env.example Dockerfile .dockerignore Gemfile Gemfile.lock \
    ~/automic_auth_api/

cd ~/automic_auth_api && bundle install
```

O `-R` não é detalhe: sem ele o `rsync` joga `deploy.yml` e `production.rb` na raiz do projeto, fora de `config/`, e o Kamal não acha nada.

O seu projeto já tinha um `config/deploy.yml`, um `Dockerfile` e um `.kamal/` — o `rails new` gera os três. Você está **substituindo** o `deploy.yml` genérico pelo nosso.

Crie o **PAT (classic)** no GitHub: **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**, com `write:packages` e `read:packages`. Ele é mostrado **uma vez**.

Confere:

```
ls config/deploy.yml .kamal/secrets .env.example # os três existem
grep -c "SERVER_IP" config/deploy.yml           # 2
```

Se der errado

Erro	Causa	Saída
<code>deploy.yml</code> foi parar na raiz	faltou o <code>-R</code>	apague e refaça o <code>rsync</code> com <code>-R</code>
<code>.kamal/</code> não veio	pasta oculta ignorada	o comando lista <code>.kamal</code> explicitamente; copie-o inteiro
<code>Could not find gem 'kamal'</code>	não rodou <code>bundle install</code>	rode
<code>git checkout aula-04</code> reclama de alterações locais	você editou o gabarito	<code>git -C ~/capacitacao-gabarito checkout .</code> — o gabarito é só leitura
o PAT sumiu da tela	é mostrado uma vez só	gere outro; não há como recuperar
criou um token <i>fine-grained</i>	o ghcr.io não aceita	tem que ser classic

Prática 4 — O `.env`

~15 minutos. Errar aqui é o que causa quase toda falha da Prática 6.

```
cd ~/automic_auth_api
cp .env.example .env
$EDITOR .env
source .env
```

Preencha: `GHCR_USER`, `SERVER_IP`, `SERVER_ARCH`, `SSH_PORT`, `APP_HOST`, `KAMAL_REGISTRY_PASSWORD` (o PAT), `RAILS_MASTER_KEY` (o conteúdo de `config/master.key`), `AUTOMIC_AUTH_API_DATABASE_PASSWORD`, `JWT_SECRET` (saída de `bin/rails secret`), `CORS_ORIGINS`, `MAILER_FROM` e os três `SMTP_*`.

Descubra a arquitetura da VM:

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'dpkg --print-architecture'
```

Confere:

```
git status          # o .env NÃO pode aparecer
bundle exec kamal config > /dev/null && echo "config ok"
```

Se o `.env` aparecer no `git status`, **pare tudo** e conserte o `.gitignore` antes de qualquer commit.

Se der errado

Erro	Causa	Saída
<code>key not found: "SERVER_IP"</code>	esqueceu o <code>source .env</code>	<code>source .env</code> — e ele vale só naquele shell
o <code>.env</code> aparece no <code>git status</code>	<code>.gitignore</code> sem a linha <code>.env</code>	acrescente antes de commitar
<code>cat: config/master.key: No such file</code>	o Rails não gerou ainda	<code>bin/rails credentials:edit</code> cria; feche o editor para salvar
<code>kamal config</code> reclama de outra variável	ela está vazia no <code>.env</code>	a mensagem diz o nome exato
você commitou o <code>.env</code> sem querer	segredo no histórico	troque todos os valores e reescreva o histórico; não basta apagar o arquivo
<code>\$EDITOR: command not found</code>	variável não definida	<code>nano .env</code> ou <code>code .env</code>

Prática 5 — O certificado e o `/etc/hosts`

~15 minutos.

```
mkdir -p tls
openssl req -x509 -newkey rsa:2048 -sha256 -days 365 -nodes \
  -keyout tls/capacita-key.pem -out tls/capacita-cert.pem \
  -subj "/CN=seunome.test" -addext "subjectAltName=DNS:seunome.test"

echo "SEU_IP seunome.test" | sudo tee -a /etc/hosts
getent hosts seunome.test
```

Confere: `getent` devolve o IP da VM. E o certificado tem o SAN certo:

```
openssl x509 -in tls/capacita-cert.pem -noout -text | grep -A1 "Subject Alternative Name"
```

Se der errado

Erro	Causa	Saída
<code>unknown option -addext</code>	OpenSSL antigo (1.0)	atualize, ou use um arquivo de config com <code>[v3_req]</code>
<code>getent</code> não devolve nada	a linha do <code>/etc/hosts</code> está errada	tem que ser <code>IP<espaço>nome</code> , sem vírgula
o navegador do Windows não acha o nome	falta a linha no hosts do Windows	<code>C:\Windows\System32\drivers\etc\hosts</code> , com o Bloco de Notas como administrador
sem "Subject Alternative Name" na saída	faltou o <code>-addext</code>	gere de novo: sem SAN, cliente moderno nenhum aceita
o <code>tls/</code> foi para o Git	não é segredo o <code>.pem</code> , mas a chave é	acrescente <code>tls/*-key.pem</code> ao <code>.gitignore</code>

Prática 6 — O primeiro deploy

~25 minutos, sendo ~5 de build. É o momento da aula.

```
source .env
bundle exec kamal config      # valida sem tocar em nada
bundle exec kamal setup      # só na primeira vez
```

Confere:

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

```
{"status":"ok","service":"automic-auth-api","environment":"production"}
```

Se der errado

Esta é a maior tabela da capacitação, e isso é de propósito: o primeiro deploy é onde tudo o que você configurou hoje é cobrado de uma vez. Se falhar, **quase nunca é o Kamal** — é uma variável do `.env`, o tipo do token ou a arquitetura. Leia a mensagem, ache a linha aqui, corrija, e rode de novo. Rodar `kamal setup` duas vezes não estraga nada.

Erro	Causa	Saída
<code>denied</code> / <code>unauthorized</code> ao empurrar a imagem	PAT sem <code>write:packages</code> , ou <i>fine-grained</i>	gere um classic com os dois escopos
<code>GHCR_USER</code> recusado pelo registry	maiúscula no nome	o ghcr.io só aceita minúsculas
<code>exec format error</code> no container	arquitetura errada	<code>dpkg --print-architecture</code> na VM e ajuste <code>SERVER_ARCH</code>
<code>Docker is not installed</code>	você rodou <code>deploy</code> no lugar de <code>setup</code>	o primeiro é sempre <code>setup</code>
<code>Host key verification failed</code>	primeira conexão do Kamal	conecte por <code>ssh</code> uma vez e aceite a chave
a aplicação sobe mas não acha o banco	<code>kamal deploy</code> não sobe accessories	<code>kamal accessory boot db</code>
<code>Connection refused</code> na 443	o proxy não está de pé, ou o <code>ufw</code> fechou	<code>ssh ... 'docker ps && sudo ufw status'</code>
<code>404</code> do proxy	<code>APP_HOST</code> diferente do nome que você chamou	os dois têm que bater exatamente
health check falhando em loop	a aplicação estoura ao subir	<code>kamal app logs -f</code> — quase sempre é <code>RAILS_MASTER_KEY</code> errada ou migration pendente
o build demora demais e o notebook trava	build de imagem consome bastante	feche o resto; é ~5 min na primeira vez

Prática 7 — O certificado, com os olhos

~10 minutos. Três comandos, e é a prática que mais ensina do dia.

```
curl https://seunome.test/api/v1/status
```

Tem que **falhar**, com `self signed certificate`. **Isso é o TLS funcionando, não falhando:** o túnel subiu; o que faltou foi a confiança.

```
curl -k https://seunome.test/api/v1/status
```

Funciona. O `-k` é "criptografa, mas não confere quem é".

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

Funciona **conferindo**. Você não desligou nada: você disse ao `curl` em quem confiar.

Abra também `https://seunome.test/api/v1/status` no navegador e leia o aviso.

Confere: escreva numa linha, para você mesmo, a diferença entre o segundo e o terceiro comando. Se conseguir, você entendeu o que uma CA é.

Se der errado

Erro	Causa	Saída
o primeiro comando funcionou	você já tinha confiado no certificado na máquina	é raro; use <code>curl</code> com <code>--cacert /dev/null</code> para ver o erro
<code>unable to get local issuer certificate</code>	mensagem diferente, mesmo fato	é a mesma lição: falta confiança
o navegador não deixa passar de jeito nenhum	HSTS de um teste anterior	use uma aba anônima, ou outro nome em <code>.test</code>

Prática 8 — O sistema inteiro, e o backup

~20 minutos. Fecha a capacitação: o que você construiu nas Aulas 2 e 3, rodando em produção.

- [] Refaça o fluxo da Aula 3 **contra a sua API no ar** — e desta vez o e-mail chega de verdade na sua caixa de entrada, não no navegador.
- [] Mude a mensagem da rota de status, commite, dê push (o CI roda) e:

```
source .env && bundle exec kamal deploy
kamal app logs -f
```

- [] Faça um backup, e confira que não está vazio:

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP \
'docker exec automic-auth-api-db pg_dump -U automic_auth_api automic_auth_api_production' \
| gzip > backup-$(date +%F).sql.gz
ls -lh backup-*.sql.gz
```

- [] `multipass stop servidor` — a VM parada não come RAM do seu notebook.

Confere: o backup tem mais que alguns bytes, e o `kamal app logs` mostrou o container novo subindo ao lado do antigo antes de trocar.

Se der errado

Erro	Causa	Saída
o e-mail não chega	SMTP não configurado, ou porta bloqueada	confira os <code>SMTP_*</code> do <code>.env</code> ; a rede pode bloquear a 587
o backup saiu com 20 bytes	o <code>pg_dump</code> falhou e o <code>gzip</code> comprimiu o vazio	tire o <code>\ gzip</code> e leia o erro
<code>docker exec</code> não acha o container	nome diferente	<code>ssh ... docker ps</code> e use o nome que aparece
o segundo deploy não mudou nada	you não commitou antes	o Kamal usa o commit atual como versão da imagem
<code>kamal deploy</code> diz que há um lock	um deploy anterior morreu no meio	<code>kamal lock release</code>

Bônus

- Derrube o app de propósito (`kamal app stop`) e veja o que o `curl` responde. Depois `kamal app boot` .
- Faça um deploy que **falha** no health check (quebre a rota `/up`) e confirme que o Kamal **não** troca o tráfego: a versão antiga continua servindo.
- Rode `kamal rollback` e veja voltar para a imagem anterior.

Recapitulando

- Um servidor é uma máquina que você opera. Que ela esteja no seu notebook ou num datacenter muda o IP, não o trabalho.
- Escolha a ferramenta do tamanho do problema. Kubernetes não era o tamanho.
- Imagem é receita, container é bolo. Multi-stage e usuário não-root.
- `clear` é configuração, `secret` é segredo — e a diferença aparece no `docker inspect` .
- Volume é o que faz o dado sobreviver ao deploy. E ainda assim não é backup.
- TLS criptografa; **CA é confiança**. São coisas separadas, e o `curl -k` mostra a costura.
- CI é o portão. Ele te avisa antes, com ou sem deploy automático.
- Numa máquina exposta, o firewall e o segredo de curta duração deixam de ser detalhe.

O que o `seem-backend` tem a mais

O que ficou de fora daqui, e por quê:

Recurso	Para quê
Datadog (logs)	logs centralizados e Error Tracking, com o app logando JSON estruturado
rack-attack	bloqueia scanner e rajada de erro por IP — a internet bate na sua porta o dia todo
Active Storage	foto de perfil, em volume Docker persistente
Solid Queue dedicado	processamento em background
Audit log	histórico de toda mutação feita por admin
Export .xlsx	relatório de presença
OpenAPI/Swagger	contrato da API documentado

Nenhum deles é difícil depois do que você viu aqui. Todos estão no `seem-backend` — agora dá para ler.